Merge Sort

Resumo completo — baseado em `Aula6_AnaliseOrdenacao.pdf` e `Lista6_AnaliseOrdenacao.pdf` (Exercícios 9, 10 e 15). Usa diretamente a teoria de 01 — Recursividade e Recorrências.

Propriedade	Valor
Estratégia	**dividir para conquistar**: divide ao meio, ordena cada metade recursivamente e **intercala**
Recorrência	$T(n) = 2T(n/2) + \Theta(n)$
Melhor / médio / pior caso	$\Theta(n \log n)$ nos três
Espaço auxiliar	$\Theta(n)$ (listas auxiliares) + $\Theta(\log n)$ de pilha
In-place	Não, na versão tradicional para vetores
Estável	Sim (usa `<=` na intercalação)

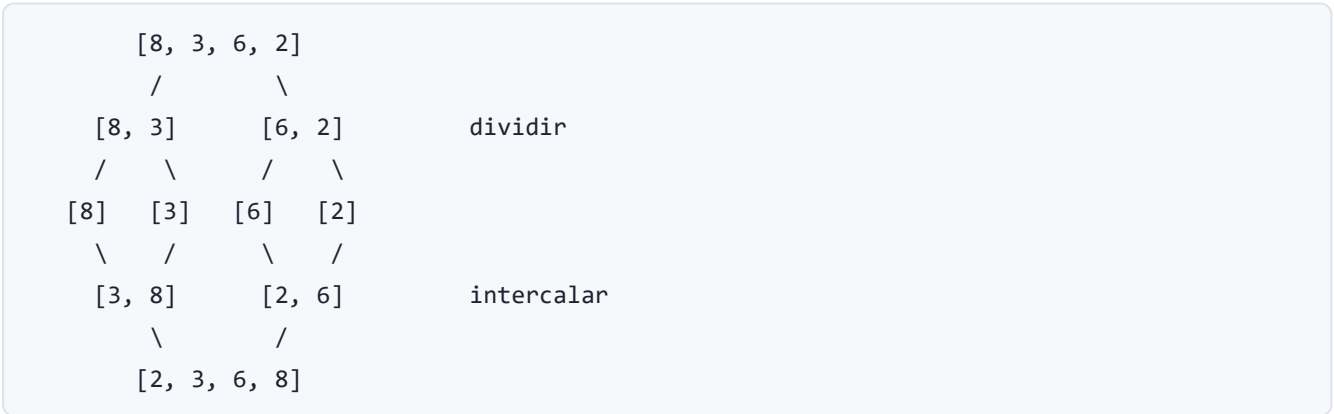
---

1. Ideia

Três etapas:

1. **Dividir** o vetor em duas partes;
2. **Ordenar recursivamente** cada parte;
3. **Intercalar** (merge) as duas partes ordenadas.

A divisão continua até subvetores de **um único elemento** (que já estão ordenados); depois, as partes são intercaladas em ordem.



Passo a passo em `[5, 2, 4, 1]`

```
dividir    [5, 2, 4, 1] -> [5, 2] | [4, 1]
dividir      [5, 2] -> [5] | [2]
intercalar   [5] + [2] -> [2, 5]
dividir      [4, 1] -> [4] | [1]
intercalar   [4] + [1] -> [1, 4]
intercalar [2, 5] + [1, 4] -> [1, 2, 4, 5]
```

---

2. Implementação

```
def merge_sort(v):
    if len(v) <= 1:                         # caso base: 0 ou 1 elemento
        return v

    meio = len(v) // 2
    esquerda = merge_sort(v[:meio])         # duas chamadas recursivas,
    direita = merge_sort(v[meio:])          # uma para cada metade

    resultado = []                          # intercalação em uma nova lista
    i = j = 0
    while i < len(esquerda) and j < len(direita):
        if esquerda[i] <= direita[j]:       # <= : em empate, a esquerda vem antes (estável)
            resultado.append(esquerda[i])
            i += 1
        else:
            resultado.append(direita[j])
            j += 1

    resultado.extend(esquerda[i:])          # sobra de uma das partes
    resultado.extend(direita[j:])
    return resultado
```

- **Caso base:** vetor com zero ou um elemento.
- **Duas chamadas recursivas:** uma para cada metade.
- **Intercalação:** percorre as duas partes ordenadas; o resultado é construído em uma **nova lista**.
- Esta implementação cria listas novas por **fatiamento** e por `resultado` → memória auxiliar proporcional a n.

---

3. A intercalação é linear

Exemplo: intercalar `[2, 5]` com `[1, 4]`.

Passo	`esquerda[i]`	`direita[j]`	Escolhe	`resultado`
1	2	1	1 (direita)	`[1]`
2	2	4	2 (esquerda)	`[1, 2]`
3	5	4	4 (direita)	`[1, 2, 4]`
fim	5	—	sobra da esquerda	`[1, 2, 4, 5]`

Por que no máximo trabalho linear? Cada comparação coloca **exatamente um** elemento em `resultado`, e nenhum elemento é examinado de novo depois de copiado. Com partes de tamanhos n_1 e n_2:

- no máximo $n_1 + n_2 - 1$ comparações (quando acaba uma parte, a outra é copiada sem comparar);
- exatamente $n_1 + n_2$ cópias.

Logo a intercalação custa $\Theta(n_1 + n_2)$.

---

4. Construindo a recorrência

A cada chamada (com n elementos):

- dois subproblemas de tamanho $\approx n/2 \rightarrow 2T(n/2)$;
- trabalho de divisão (fatiamento) + intercalação, linear $\rightarrow cn$.

$$T(n) = 2T(n/2) + cn \quad (n > 1), \qquad T(1) = d$$

5. Resolvendo a recorrência (4 métodos)

5.1 Iteração (expansão por 4 níveis)

$$\begin{aligned} T(n) &= 2T(n/2) + cn \\ &= 2\left[2T(n/4) + c\tfrac{n}{2}\right] + cn = 4T(n/4) + 2cn \\ &= 4\left[2T(n/8) + c\tfrac{n}{4}\right] + 2cn = 8T(n/8) + 3cn \\ &= 8\left[2T(n/16) + c\tfrac{n}{8}\right] + 3cn = 16T(n/16) + 4cn \\ &\vdots \\ &= 2^k\, T(n/2^k) + k\, cn \end{aligned}$$

Parada: $n/2^k = 1 \Rightarrow k = \log_2 n$ (e $2^k = n$):

$$T(n) = n\,T(1) + cn\log_2 n = dn + cn\log_2 n = \Theta(n \log n)$$

5.2 Árvore de recursão

```
Nível 0:                cn                          1 nó   -> cn
                      /    \
Nível 1:          c(n/2)    c(n/2)                    2 nós  -> cn
                  /   \      /   \
Nível 2:       c(n/4) ...  ...  c(n/4)                4 nós  -> cn
                 ...
Nível log n:   d  d  d  ...  d  d  d                  n folhas -> dn
```

Nível i	Nós	Tamanho	Custo do nível
0	1	n	cn
1	2	$n/2$	$2 \cdot c\frac{n}{2} = cn$
2	4	$n/4$	$4 \cdot c\frac{n}{4} = cn$
i	2^i	$n/2^i$	$2^i \cdot c\frac{n}{2^i} = cn$
$\log_2 n$ (folhas)	n	1	dn

- **Altura:** $\log_2 n$; **folhas:** $2^{\log_2 n} = n$.
- **Total:** $cn \cdot \log_2 n + dn = \Theta(n \log n)$.

5.3 Método Mestre

$a = 2$, $b = 2$, $f(n) = cn$; $n^{\log_2 2} = n$; $f(n) = \Theta(n)$ → **Caso 2** → $T(n) = \Theta(n \log n)$.

5.4 Substituição

A prova completa de $O(n \log n)$ e $\Omega(n \log n)$ para $T(n) = 2T(n/2) + n$ está na seção 6 de 01 — Recursividade.

5.5 Melhor = médio = pior

O Merge Sort **sempre** divide ao meio e **sempre** intercala tudo, independentemente dos dados. A ordem da entrada só altera um pouco o número de comparações de cada intercalação (entre $\min(n_1, n_2)$ e $n_1 + n_2 - 1$), mas cada nível continua custando $\Theta(n)$. Logo: $\Theta(n \log n)$ **nos três casos**.

5.6 E quando n é ímpar?

Na prática, $T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + cn$. Isso não altera a ordem assintótica:

- T é crescente. Para $2^k \leq n < 2^{k+1}$: $T(2^k) \leq T(n) \leq T(2^{k+1})$.
- $T(2^k) = \Theta(2^k \cdot k)$ e $T(2^{k+1}) = \Theta(2^{k+1}(k+1))$; como $2^k > n/2$ e $k = \lfloor \log_2 n \rfloor$, os dois limites são $\Theta(n \log n)$.
- A altura da árvore passa a ser $\lceil \log_2 n \rceil$ (diferença de no máximo 1 nível).

---

6. Memória e estabilidade

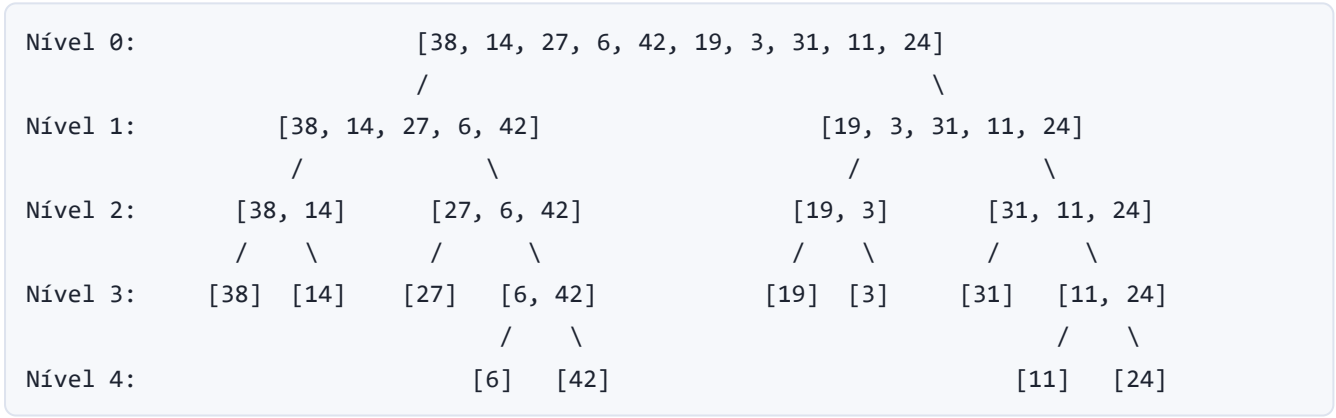
- **Listas auxiliares:** cada nível da árvore copia, no total, n elementos (fatias + `resultado`). Como as listas de um ramo já terminado são liberadas, o **pico** de memória é $\Theta(n)$ (o total alocado ao longo da execução é $\Theta(n \log n)$).
- **Pilha de recursão:** profundidade $\lceil \log_2 n \rceil \rightarrow \Theta(\log n)$.
- **Espaço auxiliar total:** $\Theta(n) \rightarrow$ **não é in-place** (existem variantes especiais in-place, mais complexas).
- **Estabilidade:** em empate (`esquerda[i] == direita[j]`), o `<=` escolhe o da **esquerda**, que veio antes na entrada → **estável**. Com `<`, perderia a estabilidade.

---

7. Exercícios resolvidos

Exercício 9 — `v = [38, 14, 27, 6, 42, 19, 3, 31, 11, 24]` ($n = 10$)

a) Árvore de recursão (divisão até subvetores unitários):



b) Primeiro nível de divisão: `[38, 14, 27, 6, 42]` e `[19, 3, 31, 11, 24]` (`meio = 10 // 2 = 5`).

c) Intercalações (de baixo para cima):

Intercalação	Resultado
`[38]` + `[14]`	`[14, 38]`
`[6]` + `[42]`	`[6, 42]`
`[27]` + `[6, 42]`	`[6, 27, 42]`
`[14, 38]` + `[6, 27, 42]`	`[6, 14, 27, 38, 42]`
`[19]` + `[3]`	`[3, 19]`
`[11]` + `[24]`	`[11, 24]`
`[31]` + `[11, 24]`	`[11, 24, 31]`
`[3, 19]` + `[11, 24, 31]`	`[3, 11, 19, 24, 31]`
`[6, 14, 27, 38, 42]` + `[3, 11, 19, 24, 31]`	**`[3, 6, 11, 14, 19, 24, 27, 31, 38, 42]`**

(No total: 19 chamadas — 10 folhas + 9 intercalações — e 23 comparações entre elementos.)

d) Folhas: 10 (uma para cada elemento).

e) Altura (raiz no nível 0, até subvetores unitários): **4** $= \lceil \log_2 10 \rceil$.

f) Intercalação linear: cada comparação move exatamente um elemento para o resultado e nenhum elemento volta a ser examinado; para partes de tamanhos n_1 e n_2, são no máximo $n_1 + n_2 - 1$ comparações e $n_1 + n_2$ cópias → $\Theta(n_1 + n_2)$ (seção 3).

Exercício 10 — recorrência e árvore

a) Significado dos termos de $T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + cn$:

- $T(\lceil n/2 \rceil)$ e $T(\lfloor n/2 \rfloor)$: custo de ordenar recursivamente a metade direita e a esquerda;
- cn: trabalho linear da divisão e da intercalação na chamada atual;
- $T(1) = d$: custo constante do caso base.

b) Com n potência de 2: $\lceil n/2 \rceil = \lfloor n/2 \rfloor = n/2$, logo $T(n) = 2T(n/2) + cn$.

c) Expansão por 4 níveis: seção 5.1 → $T(n) = 16T(n/16) + 4cn$, e em geral $2^k T(n/2^k) + kcn$.

d) Custo de cada nível: 2^i nós × $c\,n/2^i$ = cn (seção 5.2).

e) Altura: $\log_2 n$; **folhas:** n.

f) Demonstração: $T(n) = \underbrace{cn \cdot \log_2 n}_{\text{níveis internos}} + \underbrace{dn}_{\text{folhas}} = \Theta(n \log n)$. (Confirmação: Mestre Caso 2; prova formal por substituição em 01.)

g) Tamanho ímpar: as metades diferem em no máximo 1 elemento; a altura vira $\lceil \log_2 n \rceil$ e cada nível continua custando no máximo cn; limitando $T(n)$ entre $T(2^k)$ e $T(2^{k+1})$ obtém-se $\Theta(n \log n)$ (seção 5.6).

Exercício 15 — desafio integrador: `ordenar_recursivo(v)`

```
def ordenar_recursivo(v):
    if len(v) <= 1:
        return v
    meio = len(v) // 2
    esquerda = ordenar_recursivo(v[:meio])
    direita = ordenar_recursivo(v[meio:])
    return intercalar(esquerda, direita)   # linear no total de elementos
```

Vetor: `v = [47, 16, 33, 2, 29, 41, 9, 25, 6, 38]`.

a) Caso base: `len(v) <= 1`. Necessário porque (1) uma lista com 0 ou 1 elemento já está ordenada e (2) sem ele a recursão nunca termina: com 1 elemento, `meio = 0`, `v[:0]` é vazia e `v[0:]` é a **mesma** lista — chamaria `ordenar_recursivo([x])` para sempre.

b) Execução conceitual:

```
dividir [47, 16, 33, 2, 29, 41, 9, 25, 6, 38] -> [47, 16, 33, 2, 29] | [41, 9, 25, 6, 38]
  dividir [47, 16, 33, 2, 29] -> [47, 16] | [33, 2, 29]
    dividir [47, 16] -> [47] | [16]
    intercalar [47] + [16] -> [16, 47]
    dividir [33, 2, 29] -> [33] | [2, 29]
      dividir [2, 29] -> [2] | [29]
      intercalar [2] + [29] -> [2, 29]
    intercalar [33] + [2, 29] -> [2, 29, 33]
  intercalar [16, 47] + [2, 29, 33] -> [2, 16, 29, 33, 47]
  dividir [41, 9, 25, 6, 38] -> [41, 9] | [25, 6, 38]
    dividir [41, 9] -> [41] | [9]
    intercalar [41] + [9] -> [9, 41]
    dividir [25, 6, 38] -> [25] | [6, 38]
      dividir [6, 38] -> [6] | [38]
      intercalar [6] + [38] -> [6, 38]
    intercalar [25] + [6, 38] -> [6, 25, 38]
  intercalar [9, 41] + [6, 25, 38] -> [6, 9, 25, 38, 41]
intercalar [2, 16, 29, 33, 47] + [6, 9, 25, 38, 41] -> [2, 6, 9, 16, 25, 29, 33, 38, 41, 47]
```

c) Cada chamada não terminal faz **2** chamadas recursivas.

d) Recorrência: $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + cn$, $T(1) = d$ — o termo cn inclui o fatiamento (copiar n elementos) e a intercalação.

e) Resolução. Hipóteses: `intercalar` é linear; o fatiamento também é linear; n potência de 2 (ou, pela seção 5.6, isso não altera a ordem). Então $T(n) = 2T(n/2) + cn$: Mestre com $a = 2$, $b = 2$, $f(n) = cn = \Theta(n^{\log_2 2})$ → Caso 2 → $\Theta(n \log n)$. Pela árvore: $\log_2 n$ níveis de custo cn + n folhas.

f) Melhor, médio e pior caso: $\Theta(n \log n)$ (o algoritmo divide e intercala igual para qualquer entrada).

g) Memória:

- **Pilha:** profundidade $\lceil \log_2 n \rceil \rightarrow \Theta(\log n)$ (para $n = 10$: até 5 chamadas ativas, níveis 0 a 4).
- **Listas auxiliares:** fatias `v[:meio]`, `v[meio:]` e a lista de `intercalar` → $\Theta(n)$ de pico ($\Theta(n \log n)$ alocado ao longo da execução).
- **Total auxiliar:** $\Theta(n)$ → não é in-place.

h) Comparação com o Quick Sort clássico:

	Este algoritmo (Merge Sort)	Quick Sort clássico
Pior caso	$\Theta(n \log n)$	$\Theta(n^2)$
Estabilidade	estável (se `intercalar` usar `<=`)	não estável
Memória	$\Theta(n)$ listas + $\Theta(\log n)$ pilha	in-place; pilha $\Theta(\log n)$ a $\Theta(n)$

i) Como os conceitos justificam a complexidade: o **caso base** garante o término e define $T(1) = d$; o número de **chamadas recursivas** (2) e o tamanho dos subproblemas ($n/2$) dão o termo $2T(n/2)$; o trabalho local (intercalação) dá cn; a **recorrência** resume isso; e a **árvore de recursão** mostra que há $\log_2 n$ níveis de custo cn, justificando matematicamente $\Theta(n \log n)$.

Matérias correlatas

Recorrências: o Merge Sort é o "Exemplo 3" da aula de recursividade ($T(n) = 2T(n/2) + n$), resolvido por iteração, árvore, substituição e Mestre em 01 — Recursividade (seções 4–7). O Praticando 10 discute o erro de achar que "duas chamadas de metade" dá $O(n)$.

Pilha de recursão: profundidade $\Theta(\log n)$, como em `conta(n//2)` (Praticando 9) — seção 9 de 01.

Quick Sort: também divide para conquistar, mas particiona antes e não intercala depois. Veja 07 — Quick Sort.

Estabilidade e escolha do algoritmo (Ex. 8 e 14): 02 — Fundamentos.